

Data Structures & Algorithms

BCA — Semester 3 — 2024 — Answer Sheet

Subject Code: CACS201

Group B

Attempt any SIX questions. [6×5=30]

2. Differentiate between stack and queue. What are the general applications of a stack?

[2+3]

Answer: Stack vs Queue:

Basis
Stack Principle LIFO (Last-In-First-Out)
Queue Principle FIFO (First-In-First-Out)
Insertion Push — at one end (TOP)
Enqueue — at one end (REAR)
Deletion Pop — from TOP end
Dequeue — from FRONT end
Access point Only TOP element accessible
Only FRONT element accessible
Pointer movement Single pointer (top)
Two pointers (front and rear)
Overflow When $\text{top} == \text{MAX}-1$
When $\text{rear} == \text{MAX}-1$ (in linear queue)
Underflow When $\text{top} == -1$
When $\text{front} == \text{rear}$
Examples Pile of plates, browser back button
Line at ticket counter, printer queue
Variants No standard variants
Linear, circular, priority, deque

General Applications of Stack:

- 1. Expression Evaluation and Conversion:** Converting infix expressions to postfix/prefix notation and evaluating postfix expressions using a stack.
- 2. Function Call Management:** The system call stack manages function calls, local variables, return addresses, and parameters in programming languages.
- 3. Recursion:** Recursive function calls use an implicit stack to store activation records of each recursive call.
- 4. Backtracking Algorithms:** Used in maze solving, puzzle solving (e.g., Sudoku), and game playing to remember previous states and backtrack when needed.
- 5. Undo/Redo Operations:** Text editors, word processors, and graphics software use stacks to implement undo and redo functionality.
- 6. Browser History:** Web browsers use a stack to manage the back and forward navigation of visited pages.
- 7. Memory Management:** Runtime environments use stacks for dynamic memory allocation and management of local variables.
- 8. Parenthesis Matching:** Compilers use stacks to check balanced parentheses in arithmetic expressions and programming syntax.
- 9. Depth First Search (DFS):** Graph traversal algorithm uses a stack (explicit or recursive) to explore nodes depth-wise.

3. What is a linked list? How is doubly linked list different from circular linked list? Explain with example.

[1+4]

Answer: Linked List:

A linked list is a linear data structure in which elements (called nodes) are stored in non-contiguous memory locations. Each node contains two parts: **data** (the value stored) and a **pointer/reference** (address of the next node). The first node is called the **head** and the last node points to NULL, indicating the end of the list. Unlike arrays, linked lists allow dynamic memory allocation and efficient insertion/deletion at any position.

Difference between Doubly Linked List and Circular Linked List:

Basis	Doubly Linked List	Circular Linked List
Pointer structure	Each node has two pointers: next and previous	Each node has one pointer (next) like singly list
Traversal	Can be traversed in both forward and backward directions	Can be traversed only in forward direction (singly circular)
Last node pointer	Last node's next points to NULL; first node's prev points to NULL	Last node's next points back to the first node
Memory usage	More memory (extra prev pointer per node)	Less memory (same as singly linked list)
Insertion/Deletion	Easier — direct access to previous node via prev pointer	Requires traversal to find previous node
Application	Browser navigation (back/forward), undo mechanisms	CPU scheduling, round-robin algorithms, multiplayer games
NULL pointer	First and last nodes contain NULL pointers	No node contains NULL pointer (closed loop)

Example — Doubly Linked List:

NULL <-- [10|prev|next] --> [20|prev|next] --> [30|prev|next] --> NULL
Head Tail Here, node 10 has prev=NULL and next=20; node 20 has prev=10 and next=30; node 30 has prev=20 and next=NULL.

Example — Circular Linked List:

+-----+ | | \ / | [10|next] --> [20|next] --> [30|next] ---+ Head
Here, node 10 points to 20, node 20 points to 30, and node 30 points back to 10, forming a circle. There is no NULL pointer in the list.

4. What is recursion and recursive function? Write a recursive function to compute Fibonacci number.

[2+3]

Answer: Recursion:

Recursion is a programming technique in which a function calls itself directly or indirectly to solve a problem. It breaks down a complex problem into smaller, simpler subproblems of the same type. A recursive solution consists of two essential components:

1. **Base Case:** The terminating condition that stops the recursion and returns a result without making further recursive calls.
2. **Recursive Case:** The part where the function calls itself with a smaller or simpler input, moving progressively toward the base case.

Recursive Function:

A recursive function is a function that calls itself during its execution. Each recursive call creates a new activation record on the call stack containing local variables and parameters. If the base case is not properly defined, the function may result in infinite recursion leading to **stack overflow**.

Example — Recursive Function to Compute Fibonacci Number:

The Fibonacci sequence is defined as: $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$.

// C language implementation
`int fibonacci(int n) { if (n == 0) // Base case 1 return 0; if (n == 1) // Base case 2 return 1; return fibonacci(n - 1) + fibonacci(n - 2); }` // Python implementation
`def fibonacci(n): if n == 0: return 0 if n == 1: return 1 return fibonacci(n - 1) + fibonacci(n - 2)`
Execution Trace for fibonacci(5):
$$\begin{aligned} \text{fibonacci}(5) &= \text{fibonacci}(4) + \text{fibonacci}(3) = [\text{fibonacci}(3) + \text{fibonacci}(2)] + [\text{fibonacci}(2) + \text{fibonacci}(1)] \\ &= [[\text{fibonacci}(2) + \text{fibonacci}(1)] + [\text{fibonacci}(1) + \text{fibonacci}(0)]] + [[\text{fibonacci}(1) + \text{fibonacci}(0)] + 1] \\ &= [[[\text{fibonacci}(1) + \text{fibonacci}(0)] + 1] + [1 + 0]] + [[1 + 0] + 1] = [[[1 + 0] + 1] + 1] + [1 + 1] = [[1 + 1] + 1] + 2 = [2 + 1] + 2 = 3 + 2 = 5 \end{aligned}$$

5 Time Complexity: $O(2^n)$ for naive recursion (exponential)

Space Complexity: $O(n)$ for the call stack

Note: The naive recursive approach can be optimized using **memoization** or **dynamic programming** to achieve $O(n)$ time complexity.

5. How does collision occur during hashing? Explain any two hashing functions.

[3+2]

Answer: Collision in Hashing:

Collision occurs in hashing when two or more different keys are mapped to the same index (hash address) in the hash table by the hash function. Since the domain of possible keys is typically much larger than the range of hash table indices, collisions are inevitable. When a collision occurs, we need a **collision resolution technique** to store the conflicting elements.

Example of Collision:

Hash table size = 10, Hash function = $\text{key} \% 10$

- Key 25 $\rightarrow 25 \% 10 = 5$ (stored at index 5)
- Key 35 $\rightarrow 35 \% 10 = 5$ (collision! Both 25 and 35 map to index 5)

Why Collisions Occur:

1. **Limited table size:** The hash table has a finite number of slots, but the key space is potentially infinite.
2. **Non-uniform hash function:** A poorly designed hash function may cluster keys into a few indices.
3. **Birthday paradox:** Even with a good hash function, collisions become likely as the table fills up.

Two Hashing Functions:

1. Division Method (Modulo Method):

This is the simplest and most commonly used hash function. The key is divided by the table size, and the remainder is taken as the hash address.

Formula: $h(\text{key}) = \text{key} \% m$, where m is the table size.

Example: For table size $m = 10$:

- $h(42) = 42 \% 10 = 2$
- $h(73) = 73 \% 10 = 3$
- $h(128) = 128 \% 10 = 8$

Advantages: Simple, fast, easy to implement.

Disadvantages: If m is a power of 2 or has common factors with keys, clustering may occur. Best to choose m as a prime number not close to a power of 2.

2. Mid-Square Method:

In this method, the key is squared, and then a few digits from the middle of the squared result are extracted as the hash address.

Formula: $h(\text{key}) = \text{middle_digits}(\text{key}^2)$

Example: For a hash table of size 100 (need 2 digits):

- Key = 12 $\rightarrow 12^2 = 144 \rightarrow$ middle digits = 44
- Key = 45 $\rightarrow 45^2 = 2025 \rightarrow$ middle digits = 02
- Key = 68 $\rightarrow 68^2 = 4624 \rightarrow$ middle digits = 62

Advantages: Distributes keys more uniformly; less clustering compared to division method.

Disadvantages: Slightly more computation; the number of middle digits to extract must be chosen carefully based on table size.

Other Hashing Functions (for reference):

- **Folding Method:** The key is divided into parts, which are then added together to form the hash address.
- **Multiplicative Method:** $h(\text{key}) = \text{floor}(m \times (\text{key} \times A \bmod 1))$, where A is a constant between 0 and 1.

6. What is an AVL tree? Create an AVL tree from the following data: 18, 12, 14, 8, 5, 25, 34, 24, 27

[1+4]

Answer: AVL Tree:

An AVL tree (named after its inventors **Adelson-Velsky and Landis**) is a self-balancing binary search tree. In an AVL tree, the difference between the heights of the left and right subtrees (called the **balance factor**) of every node must be at most 1. The balance factor is calculated as:

Balance Factor = Height of Left Subtree – Height of Right Subtree

Valid balance factors are **-1, 0, or +1**. If the balance factor becomes less than -1 or greater than +1 after any insertion or deletion, **rotations** are performed to restore balance while maintaining the BST property.

Rotation Types:

- **LL Rotation (Right Rotation):** Performed when $BF = +2$ and left child's $BF \geq 0$
- **RR Rotation (Left Rotation):** Performed when $BF = -2$ and right child's $BF \leq 0$
- **LR Rotation (Left-Right Rotation):** Performed when $BF = +2$ and left child's $BF < 0$
- **RL Rotation (Right-Left Rotation):** Performed when $BF = -2$ and right child's $BF > 0$

AVL Tree Construction from: 18, 12, 14, 8, 5, 25, 34, 24, 27

Step 1: Insert 18

18 $BF(18) = 0$ (balanced)

Step 2: Insert 12 (left of 18)

18 / 12 $BF(18) = 1-0 = +1$ (balanced)

Step 3: Insert 14 (right of 12)

18 / 12 14 $BF(12) = 0-1 = -1$, $BF(18) = 2-0 = +2 \rightarrow$ **LR Rotation at 18**

First left rotate at 12, then right rotate at 18:

14 / 12 18 $BF(14) = 0$, $BF(12) = 0$, $BF(18) = 0$ (balanced)

Step 4: Insert 8 (left of 12)

14 / 12 18 / 8 $BF(12) = 1-0 = +1$, $BF(14) = 1-1 = 0$ (balanced)

Step 5: Insert 5 (left of 8)

14 / 12 18 / 8 / 5 $BF(8) = 1-0 = +1$, $BF(12) = 2-0 = +2$, $BF(14) = 2-1 = +1 \rightarrow$ **LL Rotation at 12**

After right rotation at 12:

14 / 8 18 / 5 12 All balanced.

Step 6: Insert 25 (right of 18)

14 / 8 18 / \ 5 12 25 $BF(18) = 0-1 = -1$, $BF(14) = 2-2 = 0$ (balanced)

Step 7: Insert 34 (right of 25)

14 / 8 18 / \ 5 12 25 34 $BF(25) = 0-1 = -1$, $BF(18) = 0-2 = -2$, $BF(14) = 2-3 = -1 \rightarrow$ **RR Rotation at 18**

After left rotation at 18:

14 / 8 25 / \ / 5 12 18 34 All balanced.

Step 8: Insert 24 (left of 25, right of 18)

14 / 8 25 / \ / 5 12 18 34 24 $BF(18) = 0-1 = -1$, $BF(25) = 2-1 = +1$, $BF(14) = 2-3 = -1$ (balanced)

Step 9: Insert 27 (right of 25? No, $27 < 34$ and $27 > 25 \rightarrow$ left of 34, right of 25)

Wait: $27 > 25$ and $27 < 34 \rightarrow$ goes to left of 34

14 / 8 25 / \ / 5 12 18 34 \ / 24 27 $BF(34) = 1 - 0 = +1$, $BF(25) = 2 - 2 = 0$, $BF(14) = 2 - 3 = -1$ (all within $\{-1, 0, 1\}$)

Wait, let me recheck: 27 is between 25 and 34, so left child of 34.

$BF(34) = 1 - 0 = +1$. $BF(25) = \text{height}(\text{left}) = 2$ (18 $\rightarrow$ 24), $\text{height}(\text{right}) = 2$ (34 $\rightarrow$ 27) = $2 - 2 = 0$. $BF(14) = \text{left height} = 2$, $\text{right height} = 3 = 2 - 3 = -1$. All balanced!

Final AVL Tree:

14 / 8 25 / \ / 5 12 18 34 \ / 24 27 All balance factors are within $\{-1, 0, +1\}$. The tree satisfies both BST property and AVL balance condition.

7. What are deterministic and non-deterministic algorithms? Explain the use of Big-Oh notation to measure the complexity of an algorithm with an example.

[2+3]

Answer: Deterministic Algorithm:

A deterministic algorithm is one that, given a particular input, will always produce the same output and follow the exact same sequence of operations. Every step has a uniquely defined next action. There is no element of randomness, choice, or uncertainty in its execution path.

Characteristics:

- Same input always yields the same output
- Execution path is predictable and repeatable
- No use of random numbers or arbitrary choices
- **Examples:** Bubble sort, insertion sort, binary search, linear search, merge sort

Non-Deterministic Algorithm:

A non-deterministic algorithm is one where the next state or output is not uniquely determined by the current state and input. It may have multiple possible execution paths, use randomness, or make arbitrary choices at certain steps. In theoretical computer science, non-deterministic algorithms are used to define the complexity class **NP** (Nondeterministic Polynomial time).

Characteristics:

- Same input may produce different outputs on different runs
- May use random numbers or oracles to make choices
- Multiple valid execution paths may exist
- **Examples:** Randomized quicksort (pivot chosen randomly), Monte Carlo algorithms, genetic algorithms, simulated annealing, non-deterministic finite automata (NFA)

Big-Oh Notation:

Big-Oh (O) notation is a mathematical notation used to describe the **upper bound** of an algorithm's growth rate as the input size n approaches infinity. It characterizes the worst-case time or space complexity, ignoring constant factors and lower-order terms. Big-Oh tells us how much the running time or memory usage grows relative to the input size.

Formal Definition:

$f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Common Big-Oh Complexities:

Complexity	Name	Example
$O(1)$	Constant	Accessing array element by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, heap sort
$O(n^2)$	Quadratic	Bubble sort, insertion sort (worst case)
$O(n^3)$	Cubic	Matrix multiplication (naive)
$O(2^n)$	Exponential	Recursive Fibonacci, subset generation

Example — Analyzing Bubble Sort with Big-Oh:

for $i = 0$ to $n-2$: for $j = 0$ to $n-i-2$: if $A[j] > A[j+1]$: swap($A[j]$, $A[j+1]$) **Step-by-step analysis:**

- The outer loop runs $(n-1)$ times $\rightarrow O(n)$
- The inner loop runs approximately $n, (n-1), (n-2), \dots, 1$ times
- Total comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 = (n^2 - n)/2$
- Ignoring constants and lower-order terms: **Time Complexity = $O(n^2)$**
- In the best case (already sorted), the inner loop still runs, so best case is also $O(n^2)$ for basic bubble sort. With optimization (flag to detect swaps), best case becomes $O(n)$.
- **Space Complexity = $O(1)$** (only a temporary variable for swapping)

Why Big-Oh is Important:

-
- Allows comparison of algorithms independent of hardware
 - Predicts scalability for large inputs
 - Focuses on asymptotic behavior (growth rate) rather than exact runtime
 - Helps in selecting the most efficient algorithm for a given problem

8. Implement the quick sort to sort the following data items: 12, 1, 14, 7, 2, 10, 4, 7, 22, 6, 15

[5]

Answer: Quick Sort Algorithm:

Quick sort is a divide-and-conquer sorting algorithm that works by selecting a 'pivot' element and partitioning the array such that all elements smaller than the pivot are placed to its left, and all elements greater than the pivot are placed to its right. The sub-arrays are then sorted recursively.

Algorithm:

Algorithm QuickSort(A, low, high): 1. If low < high: 2. pivotIndex = Partition(A, low, high) 3. QuickSort(A, low, pivotIndex - 1) 4. QuickSort(A, pivotIndex + 1, high) Algorithm Partition(A, low, high): 1. pivot = A[high] // Last element as pivot 2. i = low - 1 3. For j = low to high - 1: 4. If A[j] <= pivot: 5. i = i + 1 6. Swap A[i] and A[j] 7. Swap A[i+1] and A[high] 8. Return i + 1 **Data to Sort:** 12, 1, 14, 7, 2, 10, 4, 7, 22, 6, 15

Initial Array: [12, 1, 14, 7, 2, 10, 4, 7, 22, 6, 15]

First Call: QuickSort(A, 0, 10)

Array: [12, 1, 14, 7, 2, 10, 4, 7, 22, 6, 15], pivot = 15

Partition process (j from 0 to 9):

- j=0, A[0]=12 ≤ 15 → i=0, swap A[0],A[0] → [12, 1, 14, 7, 2, 10, 4, 7, 22, 6, 15]
- j=1, A[1]=1 ≤ 15 → i=1, swap A[1],A[1] → no change
- j=2, A[2]=14 ≤ 15 → i=2, swap A[2],A[2] → no change
- j=3, A[3]=7 ≤ 15 → i=3, swap A[3],A[3] → no change
- j=4, A[4]=2 ≤ 15 → i=4, swap A[4],A[4] → no change
- j=5, A[5]=10 ≤ 15 → i=5, swap A[5],A[5] → no change
- j=6, A[6]=4 ≤ 15 → i=6, swap A[6],A[6] → no change
- j=7, A[7]=7 ≤ 15 → i=7, swap A[7],A[7] → no change
- j=8, A[8]=22 > 15 → no swap
- j=9, A[9]=6 ≤ 15 → i=8, swap A[8] and A[9] → [12, 1, 14, 7, 2, 10, 4, 7, 6, 22, 15]

Swap A[i+1]=A[9] with A[10]: → [12, 1, 14, 7, 2, 10, 4, 7, 6, 15, 22]

Pivot index = 9

Left Sub-array: QuickSort(A, 0, 8)

Array: [12, 1, 14, 7, 2, 10, 4, 7, 6], pivot = 6

Partition (j from 0 to 8):

- j=0, 12 > 6 → no swap
- j=1, 1 ≤ 6 → i=0, swap A[0] and A[1] → [1, 12, 14, 7, 2, 10, 4, 7, 6]
- j=2, 14 > 6 → no swap
- j=3, 7 > 6 → no swap
- j=4, 2 ≤ 6 → i=1, swap A[1] and A[4] → [1, 2, 14, 7, 12, 10, 4, 7, 6]
- j=5, 10 > 6 → no swap
- j=6, 4 ≤ 6 → i=2, swap A[2] and A[6] → [1, 2, 4, 7, 12, 10, 14, 7, 6]
- j=7, 7 > 6 → no swap
- j=8, A[8]=6 is pivot position

Swap A[i+1]=A[3] with A[8]: → [1, 2, 4, 6, 12, 10, 14, 7, 7]

Wait, let me recheck: A[8] is 6 (pivot). After swap A[3] and A[8]: [1, 2, 4, 6, 12, 10, 14, 7, 7]

Pivot index = 3

Left of pivot 6: QuickSort(A, 0, 2)

Array: [1, 2, 4], pivot = 4

- j=0, 1 ≤ 4 → i=0

• $j=1, 2 \leq 4 \rightarrow i=1$

Swap $A[2]$ with $A[4]$: $\rightarrow [1, 2, 4]$ (already sorted)

Pivot index = 2

Left of 4: $[1, 2] \rightarrow \text{pivot}=2 \rightarrow \text{sorted as } [1, 2]$

Right of 4: empty

Right of pivot 6: $\text{QuickSort}(A, 4, 8)$

Array: $[12, 10, 14, 7, 7]$, $\text{pivot} = 7$

• $j=4, 12 > 7 \rightarrow \text{no swap}$

• $j=5, 10 > 7 \rightarrow \text{no swap}$

• $j=6, 14 > 7 \rightarrow \text{no swap}$

• $j=7, 7 \leq 7 \rightarrow i=4$, swap $A[4], A[7] \rightarrow [7, 10, 14, 12, 7]$

• $j=8, 7 \leq 7 \rightarrow i=5$, swap $A[5], A[8] \rightarrow [7, 7, 14, 12, 10]$

Swap $A[i+1]=A[6]$ with $A[8]$: $\rightarrow [7, 7, 7, 12, 10, 14]$

Wait, let me recount. After $j=7, i=4$. After $j=8, i=5$. Swap $A[6]$ and $A[8]$: $[7, 7, 10, 12, 14]$

Hmm, $A[6]=14, A[8]=10$. After swap: $[7, 7, 10, 12, 14]$

Pivot index = 6 (relative to full array, index = 6)

Left of this 7: $\text{QuickSort}(A, 4, 5)$

Array: $[7, 7]$, $\text{pivot} = 7 \rightarrow \text{already in place}$

Right of this 7: $\text{QuickSort}(A, 7, 8)$

Array: $[12, 14]$, $\text{pivot} = 14 \rightarrow \text{after partition: } [12, 14] \rightarrow \text{sorted}$

Right of original pivot 15: $\text{QuickSort}(A, 10, 10)$

Array: $[22] \rightarrow \text{single element, already sorted.}$

Final Sorted Array: $[1, 2, 4, 6, 7, 7, 10, 12, 14, 15, 22]$

Time Complexity: Best/Average = $O(n \log n)$, Worst = $O(n^2)$

Space Complexity: $O(\log n)$ for recursion stack

Group C

Attempt any TWO questions. $[2 \times 10 = 20]$

9. What are the differences between linear queue and circular queue? Write an algorithm to enqueue and dequeue data elements in a circular queue.

[4+3+3]

Answer: Difference between Linear Queue and Circular Queue:

Basis	Linear Queue	Circular Queue
Structure	Elements arranged in a straight line	Last position is connected back to the first position
Memory utilization	Poor — spaces at front cannot be reused after dequeue	Efficient — vacant spaces at front are reused
Overflow condition	$\text{rear} == \text{MAX} - 1$	$(\text{rear} + 1) \% \text{MAX} == \text{front}$
False overflow	Occurs when rear reaches end but front has empty spaces	Does not occur
Pointer movement	front and rear only move forward	front and rear wrap around using modulo
Implementation	Simpler	Slightly more complex
Applications	Simple scheduling, buffer management	CPU scheduling, memory management, traffic systems
Wasted space	Yes, after deletion	No wasted space (except one slot in some implementations)

Algorithm to Enqueue in Circular Queue:

Algorithm enqueue(queue, front, rear, maxSize, value):
1. If $(\text{rear} + 1) \% \text{maxSize} == \text{front}$: Print "Queue is full (Overflow)" Return
2. If $\text{front} == -1$: // First element $\text{front} = 0$
3. $\text{rear} = (\text{rear} + 1) \% \text{maxSize}$
4. $\text{queue}[\text{rear}] = \text{value}$
5. Print "Enqueued: value"
6. Return

C Implementation:

```
#define MAX 100
int queue[MAX];
int front = -1, rear = -1;
void enqueue(int value) {
    if ((rear + 1) % MAX == front) {
        printf("Queue Overflow!\n");
        return;
    }
    if (front == -1) front = 0;
    rear = (rear + 1) % MAX;
    queue[rear] = value;
    printf("Enqueued: %d\n", value);
}
```

Algorithm to Dequeue in Circular Queue:

Algorithm dequeue(queue, front, rear, maxSize):
1. If $\text{front} == -1$: Print "Queue is empty (Underflow)" Return -1
2. $\text{value} = \text{queue}[\text{front}]$
3. If $\text{front} == \text{rear}$: // Only one element $\text{front} = -1$ $\text{rear} = -1$
4. Else: $\text{front} = (\text{front} + 1) \% \text{maxSize}$
5. Print "Dequeued: value"
6. Return value

C Implementation:

```
int dequeue() {
    int value;
    if (front == -1) {
        printf("Queue Underflow!\n");
        return -1;
    }
    value = queue[front];
    if (front == rear) {
        front = -1;
        rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
    printf("Dequeued: %d\n", value);
    return value;
}
```

Time Complexity: Enqueue = $O(1)$, Dequeue = $O(1)$

Space Complexity: $O(\text{maxSize})$ for the queue array

10. What is B-tree? How insertions and deletions of elements can be done in a B-tree?

[2+8]

Answer: B-Tree:

A B-tree is a self-balancing multi-way search tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. It is a generalization of binary search trees where a node can have more than two children. B-trees are particularly well-suited for storage systems that read and write large blocks of data, such as databases and file systems.

Properties of B-tree of order m :

1. Every node has at most m children.
2. Every internal node (except root) has at least $\lceil m/2 \rceil$ children.
3. The root has at least 2 children if it is not a leaf node.
4. All leaf nodes are at the same depth (balanced).
5. A node with k children contains $k-1$ keys.
6. Keys within a node are stored in ascending order.

Insertion in B-Tree:

The insertion algorithm ensures that all B-tree properties are maintained after adding a new key.

Algorithm BTreeInsert(T , key): 1. If T is empty: Create a new root node containing key Return 2. Find the appropriate leaf node where key should be inserted 3. Insert key into the leaf node in sorted order 4. If leaf node overflows (has m keys, i.e., $m-1$ keys + 1 new = m keys which exceeds $m-1$ limit): a. Split the node at median key b. Move median key up to parent node c. Create two new nodes: left with keys $<$ median, right with keys $>$ median d. If parent overflows, recursively split up to root 5. If root splits, create a new root with the median key and increase tree height by 1

Example of Insertion in B-Tree of order 3 (2-3 tree):

Insert 10, 20, 5, 6, 12, 30, 7, 17

- Insert 10: [10]
- Insert 20: [10, 20]
- Insert 5: [5, 10, 20] → Overflow! Split at 10
New root: [10], Left: [5], Right: [20]
- Insert 6: Goes to [5] → [5, 6]
- Insert 12: Goes to [20] → [12, 20]
- Insert 30: Goes to [12, 20] → [12, 20, 30] → Overflow! Split at 20
Root [10] becomes [10, 20], Left of 20: [12], Right of 20: [30]
- Insert 7: Goes to [5, 6] → [5, 6, 7] → Overflow! Split at 6
Parent [10, 20] becomes [6, 10, 20] → Overflow! Split at 10
New root: [10], Left: [6], Right: [20], with appropriate children
- Continue similarly for remaining keys.

Deletion in B-Tree:

Deletion requires ensuring that nodes do not underflow (drop below minimum key count).

Algorithm BTreeDelete(T , key): 1. Locate the node containing key 2. If key is in a leaf node: a. Simply remove key b. If node underflows (keys $<$ $\lceil m/2 \rceil - 1$): - Try to borrow a key from left or right sibling - If siblings have minimum keys, merge with a sibling and move separator key down from parent - If parent underflows, recursively fix up to root 3. If key is in an internal node: a. Replace key with its inorder predecessor (largest in left subtree) or inorder successor (smallest in right subtree) b. Recursively delete the predecessor/successor from the leaf 4. If root becomes empty after merge, make the merged node the new root

Cases for Handling Underflow during Deletion:

Case 1: The key to be deleted is in a leaf and the leaf has more than the minimum number of keys. Simply delete it.

Case 2: The key is in a leaf but the leaf has the minimum number of keys.

- **2a:** If a sibling has more than minimum keys, borrow a key from the sibling through the parent (rotation).
- **2b:** If both siblings have minimum keys, merge the leaf with a sibling and pull down the separating key from the parent.

Case 3: The key is in an internal node. Replace it with its inorder predecessor or successor and delete that key from the leaf.

Time Complexity: Search, Insert, Delete = $O(\log_m n)$ where m is order and n is number of keys.

Space Complexity: $O(n)$

11. Explain the different ways to represent a graph. For the following graph use Prim's algorithm to find a minimum spanning tree starting from the node 'A'.

[5+5]

Answer: Ways to Represent a Graph:

1. Adjacency Matrix:

An adjacency matrix is a 2D array of size $V \times V$ (where V is the number of vertices). For an unweighted graph, $\text{matrix}[i][j] = 1$ if there is an edge from vertex i to vertex j , and 0 otherwise. For a weighted graph, $\text{matrix}[i][j] = \text{weight of the edge}$, or $\infty/0$ if no edge exists.

Example (undirected weighted graph with vertices A, B, C, D):

ABCD A0420 B4015 C2103 D0530 **Advantages:** Simple, edge lookup in $O(1)$.

Disadvantages: $O(V^2)$ space, wasteful for sparse graphs.

2. Adjacency List:

An adjacency list represents a graph as an array of linked lists. Each vertex has a linked list of its adjacent vertices (with weights for weighted graphs).

Example (same graph):

$A \rightarrow (B, 4) \rightarrow (C, 2)$ $B \rightarrow (A, 4) \rightarrow (C, 1) \rightarrow (D, 5)$ $C \rightarrow (A, 2) \rightarrow (B, 1) \rightarrow (D, 3)$ $D \rightarrow (B, 5) \rightarrow (C, 3)$

Advantages: Space-efficient $O(V + E)$, efficient for sparse graphs.

Disadvantages: Edge lookup takes $O(\text{degree})$ time.

3. Edge List:

An edge list stores all edges as a list of tuples (u, v, weight) for weighted graphs.

Example: $[(A,B,4), (A,C,2), (B,C,1), (B,D,5), (C,D,3)]$

4. Incidence Matrix:

An incidence matrix is a $V \times E$ matrix where $\text{matrix}[i][j] = 1$ if vertex i is incident to edge j , and 0 otherwise. Used less frequently.

Prim's Algorithm for Minimum Spanning Tree (MST):

Prim's algorithm finds the MST of a weighted, connected, undirected graph. It starts from an arbitrary vertex and grows the tree by adding the cheapest edge that connects a vertex in the tree to a vertex outside the tree.

Algorithm:

Algorithm Prim(G, start): 1. Initialize a set $\text{MST} = \{\text{start}\}$ 2. Initialize $\text{totalWeight} = 0$ 3. While MST does not include all vertices: 4. Find the minimum weight edge (u, v) such that $u \in \text{MST}$ and $v \notin \text{MST}$ 5. Add v to MST 6. Add edge (u, v) to MST edges 7. $\text{totalWeight} += \text{weight}(u, v)$ 8. Return MST edges and totalWeight **Note:** Since the specific graph edges were not fully visible in the original paper, we use a representative graph commonly used in TU BCA exams:

A / | 2 4 3 / | C --- B --- D 1 5 Edges: A-C=2, A-B=4, A-D=3, B-C=1, B-D=5

Step-by-Step Execution starting from 'A':

Step	MST Vertices	Candidate Edges	Min Edge	Selected	Weight
1	{A}	A-C(2), A-B(4), A-D(3)	A-C	2	2
2	{A, C}	A-B(4), A-D(3), C-B(1)	C-B	1	3
3	{A, C, B}	A-D(3), B-D(5)	A-D	3	6
4	{A, C, B, D}				

Total MST Weight = 2 + 1 + 3 = 6

MST Edges: (A, C, 2), (C, B, 1), (A, D, 3)

Final MST Structure:

A / 2 3 / C---1---B D is connected via A **Time Complexity:** $O(E \log V)$ using a binary heap, or $O(E + V \log V)$ using a Fibonacci heap.

Space Complexity: $O(V)$

■ *Diagram Required:* Graph adjacency representations and Prim's algorithm MST should be drawn showing vertices, edges, and weights.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.